

Checking Failure-Path Telemetry in the Data a Project Already Emits

Halley Young

Preprint, October 2026. Not peer reviewed.

Abstract—When a request fails in production, responders need the emitted telemetry to say which request failed and why. We present **TELEMETRY-CONTRACTS**, a pure-Python command-line tool that checks this property against logs, traces and metrics a project already has. It needs no contract written in advance and no instrumentation convention. It defines six gap classes with written definitions: failure without correlation id, failure without error evidence, span without duration, raw sensitive value, unclassified identifier, and unbounded metric label. Each check is a deterministic function of the events. Our claim is that the two failure-path classes can be checked usefully in this setting. On 60 findings sampled from a scan of 58 public repositories (116,420 events), the failure-path checks matched their definitions in 13/15 and 12/15 cases. The privacy checks did worse, at 9/15 and 1/15 (10/15 counting borderline session-id cases). Only 21 of the 60 sampled findings came from logs the project itself wrote, and 7 were judged worth fixing. An LLM, not a human expert, labelled this sample. On a 112-item gold set written by the tool’s author, the tool reaches F1 0.953. Two zero-shot LLMs given the same class definitions reach 0.991 (Claude Haiku 4.5) and 0.952 (GPT-4.1-mini), and neither difference is significant. The tool’s advantage over the LLMs is therefore operational. It runs offline and deterministically at no cost per event, and it has no accuracy advantage on this set. All prompts, raw model outputs, corpus findings and labels are released.

I. INTRODUCTION

Logging research has long studied whether code logs enough to diagnose failures. It has measured logging practice [1], [2], [3], added diagnostic variables to existing log statements [4], added log statements on error paths [5], and predicted where developers should log [6], [7], [8]. These approaches act on source code. They answer the question “should this code location log?”

This paper asks a narrower question about the output. Given the telemetry a service has emitted, can a responder join each failure event to a request, and does each failure carry structured evidence of its cause? Distributed tracing makes the first property routine in principle [9]. In practice teams emit heterogeneous JSON logs, logfmt lines and OTLP exports, under field names that follow no single convention. The OpenTelemetry semantic conventions [10] standardise names but only help teams that have adopted them.

TELEMETRY-CONTRACTS treats each such property as a contract clause over a finite set of events and reports violations. The checks run on whatever a project ships, so they can be applied to repositories whose authors never used the tool. The contribution of this paper is the tool plus an evaluation that includes real LLM baselines and a validated corpus study. The tool also implements contract inference, temporal properties,

refinement checking and a browser playground. Those features are described in the repository and are not evaluated here.

We address three questions:

- **RQ1.** How accurate are the checks on a labelled set of telemetry samples?
- **RQ2.** How do they compare with rule baselines and with zero-shot LLMs given the same definitions?
- **RQ3.** When run on public repositories, how often are the findings correct and worth fixing?

II. GAP CLASSES AND CHECKS

An *event* is a JSON object after format normalisation (Section III). A *failure event* has severity error, critical or fatal, or a status/outcome field whose value denotes failure. The checks are:

- **missing-correlation:** A failure event with no correlation identifier (`trace_id`, `traceId`, `request_id`, `span_id` or a recognised alias).
- **missing-error-evidence:** A failure event with no error code, exception type or status field.
- **missing-duration:** A span event with no duration or latency measure.
- **sensitive-values:** A scalar value matching a raw-secret or PII pattern: email, bearer token, JWT, password assignment, or a card or phone number in a field with a card or phone name.
- **unclassified-sensitive:** A field whose name denotes a tenant, customer, account or user identifier, or another sensitive attribute, emitted without redaction, hashing or a privacy classification.
- **unbounded-cardinality:** A metric label whose distinct-value count is high relative to the sample, reported only above a minimum sample size.

The first three checks are presence checks over field names. A reported absence is only as good as the alias table. A correlation id under an unrecognised key causes a false positive, and Section IV-C shows that nested keys cause the same problem. The last three checks are pattern or statistical heuristics tuned for precision. Every finding records its source file, source line, check code and a remediation hint, and can be exported as SARIF for code-scanning pipelines.

III. IMPLEMENTATION

TELEMETRY-CONTRACTS is a standard-library-only Python package with a command-line interface and a GitHub Action. Adapters read JSONL, JSON arrays, logfmt and

OTLP/JSON collector exports. They map common aliases (level/severity, msg/message, service.name, traceId) and infer the signal kind with a confidence score. The `scan-repo` command clones a repository at a commit, discovers candidate telemetry files with a bounded walk that skips configuration, lock files and vendored directories, and checks each file. For each check code it keeps at most 25 example findings per file, so per-file counts are lower bounds. The `corpus-run` command scans a pinned-commit manifest with a content-addressed cache. Every aggregate uses integer or fixed-point arithmetic, so a rerun on the same inputs is byte-identical. The repository’s CI checks this against a SHA-256 manifest of 13 generated artifacts.

IV. EVALUATION

All numbers below are regenerated by the commands in `REPRODUCE.md`. Raw outputs are under `results/` and `reports/`.

A. RQ1: Gold-set accuracy

Setup. The gold set (`benchmarks/ground_truth/curated.jsonl`) has 112 (sample, class) pairs, 54 positive and 58 negative, covering all six classes. Each label records whether the class is present according to the meaning of its definition, independent of the detector’s field lists. Five *hard* items were written to probe known blind spots, such as a correlation id under the key `correlationGuid` and a base64-encoded credential. *All labels were written by one person, the tool’s author.* An earlier version of the set carried 17 “second labels”, but the same author had written them in the generator script, so they did not measure agreement. We removed them, and we report no inter-rater agreement.

Results. The tool scores precision 0.962 (51/53, Wilson 95% CI 0.87–0.99), recall 0.944 (51/54, 0.85–0.98) and F1 0.953. Per-class F1 ranges from 0.923 (unbounded-cardinality) to 1.000 (missing-duration). The five errors are exactly the five hard items: two false positives (an unrecognised correlation key, and error evidence present only in free text) and three false negatives (a base64 secret, an unrecognised tenant field, and a cardinality sample below the trust threshold). The author wrote the hard items knowing these blind spots, so RQ1 shows the checks behave as specified on a set built around their specification. It does not show accuracy on independent data. RQ3 addresses that.

B. RQ2: Rule baselines and LLMs

Setup. Every method receives only the events and the class name, never the label, and is scored by the same code. *rule-light* checks only for obvious field names. *semconv-only* flags missing OpenTelemetry convention fields, and its three classes without a convention field are reported out of scope. For the LLM rows we sent each item once through OpenRouter to `anthropic/claude-haiku-4.5` and `openai/gpt-4.1-mini`, at temperature 0 with at most 200 output tokens. The prompt states the class name, its definition and the events as JSON, and asks for YES or NO

TABLE I
GOLD SET (N = 112). COVERED-CLASS F1 EXCLUDES CLASSES A METHOD CANNOT EXPRESS. DISCORDANT PAIRS ARE COUNTED AGAINST THE TOOL: ITEMS ONLY THE TOOL GETS RIGHT / ITEMS ONLY THE BASELINE GETS RIGHT.

Method	P	R	F1	Cov. F1	Disc. (p)
TELEMETRY-CONTRACTS	.962	.944	.953	.953	–
rule-light	.857	.667	.750	.809	20/1 (<.001)
semconv-only	.628	.500	.557	.771	38/0 (<.001)
Claude Haiku 4.5	.982	1.00	.991	.991	0/4 (.125)
GPT-4.1-mini	.980	.926	.952	.952	3/3 (1.00)

plus one sentence. It contains no examples. All 224 responses parsed. The raw responses, served model ids, token counts and costs are in `results/llm_baseline/`. Both runs together cost US\$0.077. The comparison replays the recorded verdicts offline. Significance is an exact two-sided McNemar test on the paired per-item outcomes.

Results. Table I shows the results. The tool beats both rule baselines significantly. It does not beat either LLM. Claude Haiku 4.5 is correct on every item the tool gets right, and on four of the five hard items the tool misses. The fifth hard item, error evidence present only in free text, is labelled negative in the gold set, and every method including both LLMs flags it. GPT-4.1-mini fixes three hard items. It also misses three unclassified-identifier positives that the tool catches. Neither difference is significant at $n = 112$, and the confidence intervals are wide. The hard items probe exactly the field-keyed incompleteness that a model reading the definition semantically avoids. We therefore claim no accuracy advantage over LLMs. The tool’s remaining advantages are operational. It runs offline with no API key, gives the same output on every run, and has no per-item cost. On a repository scan it would otherwise need one model call per (event sample, class) pair: about 300 prompt tokens per item on this set, before any batching.

C. RQ3: Corpus study with validated findings

Setup. The corpus manifest pins 59 public repositories at full commit SHAs. One has since been deleted upstream. The other 58 are on GitHub (56), GitLab (1) and Bitbucket (1). They were selected by an earlier curation script that searched for repositories shipping parseable telemetry files and kept those that cloned and parsed. They are mostly small projects: 34 are primarily Python, and the rest include Go, Rust, TypeScript and Java. We downloaded the 58 and ran `corpus-run`, which took 8.5 minutes on a laptop. We then extracted all 2,269 findings with their source events. From these we drew a stratified sample of 15 findings per class that fired, with at most two per (repository, class), ordered by a hash.

Validation procedure. Each sampled finding was inspected by the LLM agent that ran the study. The agent read the flagged event in the pinned checkout and the class definition, then assigned three labels: whether the event matches the definition (yes/no/borderline), what kind of file it is (runtime

TABLE II

RQ3: CORPUS PREVALENCE OVER 56 TELEMETRY-BEARING REPOSITORIES, AND VALIDATED PRECISION ON 15 SAMPLED FINDINGS PER CLASS (WILSON 95% CI). BORDERLINE (SESSION-ID) CASES COUNT AS INCORRECT IN THE STRICT COLUMN.

Class	Repos	Findings	Matches def.	Worth fixing
missing-correlation	10	171	13/15 (.62-.96)	2/15
missing-error-ev.	10	127	12/15 (.55-.93)	4/15
sensitive-values	10	110	9/15 (.36-.80)	1/15
unclassified-sens.	19	1,861	1/15 (+9 bord.)	0/15
All sampled			35/60 (.46-.70)	7/60

log written by the project, sample telemetry, test fixture, or non-telemetry), and whether a maintainer would plausibly treat it as a defect in telemetry the project emits. *This is LLM-assisted inspection with a single rater. It is not human expert review.* The labels and a one-line rationale for each are in `results/rq3/validation_labels.csv`. A blinded packet for an independent human rater is in `docs/evaluation/annotation/`.

Results. Table II summarises the scan and the validation. The scan found no missing-duration or unbounded-cardinality findings. Of the 14 telemetry-bearing repositories with at least one failure event, 10 have a failure event without a recognised correlation id.

The failure-path checks were mostly correct by their definitions: 25 of 30 sampled findings matched. The five errors fall into three patterns. Three come from Java logback events that hold `trace_id` under a nested `mdc` object (one finding) or the exception class under a nested `throwable` object (two findings). One is an OTLP record carrying the convention attribute `error.type`, which the error-evidence check does not credit. The last is a disaster-recovery drill specification that was parsed as a failure event because of its category name. The first two patterns can be fixed by adding the aliases and descending into nested objects. The third is a file-discovery problem.

The privacy checks were weak on this corpus. Six of the 15 sensitive-value findings came from regexes firing on prose: “Change Password:”, “opaque-bearer shortcuts”, and placeholder credentials in code-review datasets. Five of the 15 unclassified-identifier findings were token *counts* that matched the `token` name heuristic, and nine were `session_id` fields whose sensitivity depends on context.

File discovery limits how many findings are useful, across all classes. Only 21 of the 60 sampled findings came from runtime logs the project wrote. The rest came from demo telemetry (22), test fixtures (5) and files that are not telemetry at all (12), such as LLM preference datasets and issue trackers stored as JSONL. A finding in a demo file can be correct by definition and still be of no use to the maintainer. Seven of the 60 were judged worth fixing: four committed error-log entries with no exception type or error code, two failures without request ids, and one committed agent prompt log containing the author’s personal email address.

Answer. On real repositories the failure-path checks are usable as written (25/30 sampled findings correct), with two fixable alias gaps. The privacy name heuristics and the file-discovery step are not yet precise enough to report as defects without review. The repository counts in Table II include false positives and non-runtime files, and the finding counts are capped at 25 per check per file, so neither column is a prevalence estimate.

V. THREATS TO VALIDITY

Labels. RQ1 and RQ2 use one author’s labels on items the author constructed, and RQ3 uses one LLM rater. Neither has been checked by an independent human. The packet and agreement script for that check are released, and the check is outstanding. The LLMs in RQ2 may share failure modes with the LLM rater in RQ3. The two tasks use different inputs and questions, but we cannot rule out correlated judgements.

Sample sizes. With 15 findings per class, the intervals in Table II are 0.34 to 0.44 wide. The McNemar tests in RQ2 have little power at $n = 112$ with few discordant pairs.

LLM baseline. We used one zero-shot prompt per model with no tuning, and a single recorded sample. Re-querying at temperature 0 may give different answers. The committed log is the record of this run.

Corpus. The corpus was selected for parseable telemetry, not as a random sample of software projects. It is dominated by small Python projects and contains many demo and fixture files, so prevalence does not generalise.

VI. RELATED WORK

Logging practice and log enhancement. Empirical studies characterise how and where developers log in C/C++, Java, industrial and mobile code [1], [2], [6], [3], and what logging costs and buys [11]. LogEnhancer adds causally relevant variables to existing log statements [4]. Errlog inserts log statements at exception and error sites [5]. Learning-based approaches recommend where to log [7], [8], and LLM-based approaches generate the statements themselves [12], [13]. Chen and Jiang detect anti-patterns in logging code [14]. All of these analyse or change source code. TELEMETRY-CONTRACTS examines the emitted output instead, so it can check repositories in any language without building them. It cannot say which code location should have logged.

Log analysis. Automated log analysis parses, mines and detects anomalies in logs once they exist [15], [16], and Loghub provides datasets for that work [17]. These techniques assume the logs are adequate. TELEMETRY-CONTRACTS checks whether they are adequate for specific incident questions. Loghub contains raw system logs without labels for the gap classes used here, so it does not provide independent ground truth for RQ1.

Specification mining and runtime verification. Specification miners infer temporal or invariant specifications from traces [18], [19], [20]. Runtime verification checks traces against temporal-logic specifications [21], [22], and hyper-properties capture information-flow requirements over sets of

traces [23]. TELEMETRY-CONTRACTS’s contract inference and temporal clauses build on these ideas, but the checks evaluated here are simpler per-event properties. We do not evaluate the inference.

Conventions and scanners. The OpenTelemetry semantic conventions [10] define field names, and our semconv-only baseline approximates a conformance linter. Secret scanners such as detect-secrets and gitleaks look for credentials in source files. The sensitive-value check applies similar patterns to telemetry values, and RQ3 shows it inherits their false positives on prose.

VII. CONCLUSION AND REMAINING WORK

TELEMETRY-CONTRACTS checks failure-path telemetry in data projects already emit, without configuration. On public repositories its correlation and error-evidence checks matched their definitions in 25 of 30 sampled findings. On an author-built gold set it is as accurate as small zero-shot LLMs, but not more accurate. Remaining work: (1) independent human annotation of the gold set and the RQ3 sample using the released packet; (2) nested-key and `error.type` support, then a re-run of RQ3; (3) restricting file discovery to runtime telemetry, or reporting artifact kind, so findings in fixtures and datasets are separated; (4) reporting at least one finding upstream and recording the maintainers’ responses.

ARTIFACT AVAILABILITY

Source, gold set, corpus manifest, LLM prompts and raw outputs, corpus findings and validation labels: <https://github.com/thehalleyyoung/telemetry-contract-semantics>. See `REPRODUCE.md`.

REFERENCES

- [1] D. Yuan, S. Park, and Y. Zhou, “Characterizing logging practices in open-source software,” in *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, 2012, pp. 102–112.
- [2] B. Chen and Z. M. J. Jiang, “Characterizing logging practices in Java-based open source software projects – a replication study in Apache Software Foundation,” *Empirical Software Engineering*, vol. 22, no. 1, pp. 330–374, 2017.
- [3] Y. Zeng, J. Chen, W. Shang, and T.-H. P. Chen, “Studying the characteristics of logging practices in mobile apps: A case study on F-Droid,” *Empirical Software Engineering*, vol. 24, no. 6, pp. 3394–3434, 2019.
- [4] D. Yuan, J. Zheng, S. Park, Y. Zhou, and S. Savage, “Improving software diagnosability via log enhancement,” in *Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2011, pp. 3–14.
- [5] D. Yuan, S. Park, P. Huang, Y. Liu, M. M. Lee, X. Tang, Y. Zhou, and S. Savage, “Be conservative: Enhancing failure diagnosis with proactive logging,” in *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2012, pp. 293–306.
- [6] Q. Fu, J. Zhu, W. Hu, J.-G. Lou, R. Ding, Q. Lin, D. Zhang, and T. Xie, “Where do developers log? An empirical study on logging practices in industry,” in *Companion Proceedings of the 36th International Conference on Software Engineering (ICSE Companion)*, 2014, pp. 24–33.
- [7] J. Zhu, P. He, Q. Fu, H. Zhang, M. R. Lyu, and D. Zhang, “Learning to log: Helping developers make informed logging decisions,” in *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2015, pp. 415–425.
- [8] Z. Li, T.-H. P. Chen, and W. Shang, “Where shall we log? Studying and suggesting logging locations in code blocks,” in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2020, pp. 361–372.
- [9] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan, and C. Shanbhag, “Dapper, a large-scale distributed systems tracing infrastructure,” Google, Inc., Tech. Rep. dapper-2010-1, 2010.
- [10] OpenTelemetry Authors, “OpenTelemetry semantic conventions,” <https://opentelemetry.io/docs/specs/semconv/>, accessed 2026-10-06.
- [11] H. Li, W. Shang, B. Adams, M. Sayagh, and A. E. Hassan, “A qualitative study of the benefits and costs of logging from developers’ perspectives,” *IEEE Transactions on Software Engineering*, vol. 47, no. 12, pp. 2858–2873, 2021.
- [12] J. Xu, Z. Cui, Y. Zhao, X. Zhang, S. He, P. He, L. Li, Y. Kang, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, “UniLog: Automatic logging via LLM and in-context learning,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024, pp. 14:1–14:12.
- [13] Y. Li, Y. Huo, Z. Jiang, R. Zhong, P. He, Y. Su, L. Briand, and M. R. Lyu, “Exploring the effectiveness of LLMs in automated logging generation: An empirical study,” 2023.
- [14] B. Chen and Z. M. J. Jiang, “Characterizing and detecting anti-patterns in the logging code,” in *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2017, pp. 71–81.
- [15] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, “A survey on automated log analysis for reliability engineering,” *ACM Computing Surveys*, vol. 54, no. 6, pp. 130:1–130:37, 2022.
- [16] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, and M. R. Lyu, “Tools and benchmarks for automated log parsing,” in *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 121–130.
- [17] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, “Loghub: A large collection of system log datasets for AI-driven log analytics,” in *Proceedings of the 34th IEEE International Symposium on Software Reliability Engineering (ISSRE)*, 2023, pp. 355–366.
- [18] G. Ammons, R. Bodík, and J. R. Larus, “Mining specifications,” in *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2002, pp. 4–16.
- [19] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, “The Daikon system for dynamic detection of likely invariants,” *Science of Computer Programming*, vol. 69, no. 1–3, pp. 35–45, 2007.
- [20] C. L. Goues and W. Weimer, “Specification mining with few false positives,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. LNCS, vol. 5505, 2009, pp. 292–306.
- [21] M. Leucker and C. Schallhart, “A brief account of runtime verification,” *The Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.
- [22] A. Bauer, M. Leucker, and C. Schallhart, “Runtime verification for LTL and TLTL,” *ACM Transactions on Software Engineering and Methodology*, vol. 20, no. 4, pp. 14:1–14:64, 2011.
- [23] M. R. Clarkson and F. B. Schneider, “Hyperproperties,” *Journal of Computer Security*, vol. 18, no. 6, pp. 1157–1210, 2010.